PneumoniaMNIST: SVM vs. CNN

by Matteo Lugli

Based on the MedMNIST benchmark

Abstract

This individual project for the Computational and Statistical Learning course at UniMoRe compares classical feature engineering and learned representations for binary classification on PneumoniaMNIST. The benchmark contains 5,856 grayscale chest X-ray images at 28×28 resolution, with fixed training, validation, and test splits. Two support vector machine pipelines, using principal component analysis (PCA) and histograms of oriented gradients (HOG), are compared with a compact convolutional neural network (CNN) trained using stochastic gradient descent with momentum or Adam. The study examines discrimination, threshold-based classification, probability quality, training behavior, and computational cost. It also considers changes to the SGD learning rate and the use of random seeds.

This report develops the mathematical background, explains the Python implementation, and reconstructs the experiments from the original 42-slide presentation. The reported seeded CNN runs reach ROC AUC values of 0.9660 (SGD) and 0.9644 (Adam), compared with approximately 0.955 for SVM with HOG. Adam has the highest reported accuracy and F1 score and the lowest Brier score among these configurations. These are historical, rounded presentation values, not newly reproduced measurements or averages across repeated runs. The source review identifies limitations in checkpoint selection, cross-validation preprocessing, and reproducibility. The publication version fixes checkpoint copying and adds explicit run controls while retaining the original script in an archive. The findings support the potential of compact CNNs on this benchmark, with conclusions limited by the available experimental evidence. This is an educational classification study, not a clinically validated diagnostic system.

1 Project Objective and Scope

The project asks whether a small CNN learning its own image representation can improve on an RBF-kernel SVM trained on explicit image features. A second question concerns optimization: how do SGD with momentum and Adam behave under different learning rates and random initializations?

Four principal configurations are considered: SVM with PCA features, SVM with HOG features, CNN with SGD and momentum, and CNN with Adam. The experiment extends the course topics of empirical risk minimization, kernel methods, regularization, stochastic optimization, and deep learning. The original presentation (original slides) is the source of the historical results; the supplied Python script is used to establish the implementation details.

The task is binary classification, with normal images labeled 0 and pneumonia images labeled 1. It does not address localization, disease severity, clinical decision making, or deployment on full-resolution radiographs.

2 Dataset and Evaluation Protocol

2.1 PneumoniaMNIST

PneumoniaMNIST is a subset of MedMNIST based on pediatric chest X-ray images. MedMNIST center-crops and resizes the source images to a single-channel 28×28 representation and provides standardized splits (MedMNIST; dataset metadata). The project uses these existing splits without constructing a new random train/test partition.

Split	Normal	Pneumonia	Total
Training	1,214	3,494	4,708
Validation	135	389	524
Test	234	390	624
Total	1,583	4,273	5,856

Table 1: Class counts verified from the supplied local MedMNIST archive. The split sizes agree with the official metadata.

Contrary to the description in slide 3, the data are not approximately balanced: pneumonia represents about 74.2% of training images and 62.5% of test images. This makes accuracy alone an incomplete summary. A classifier always predicting pneumonia would achieve 62.5% test accuracy and an F1 score of approximately 0.769, despite having no discrimination ability.

The code uses `ToTensor()` to map pixel intensities to $[0, 1]$. No data augmentation, external pretraining, or class weighting is configured. The SVM pipelines additionally standardize their input features.

2.2 Metrics

Let p_i be the predicted probability of pneumonia and $\hat{y}_i = \mathbb{1}[p_i \geq 0.5]$ the predicted class. The same threshold is used for the reported accuracy and F1 scores.

ROC AUC. The primary metric measures ranking quality across thresholds. Informally,

$$\mathrm{AUC} = \Pr(p^+ > p^-) + \tfrac{1}{2}\Pr(p^+ = p^-),$$

where p^+ and p^- are scores for positive and negative examples. AUC does not evaluate whether a threshold of 0.5 is appropriate.

Accuracy and F1.

$$\text{Accuracy} = \frac{TP + TN}{N}, \qquad F_1 = \frac{2TP}{2TP + FP + FN}.$$

F1 summarizes precision and recall for the positive class. Both metrics depend on the decision threshold and on the distribution of classes.

Brier score.

$$\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(p_i - y_i)^2.$$

Lower values indicate better probabilistic predictions. The Brier score reflects both reliability and discrimination; a lower value by itself is not proof of better calibration. Reliability diagrams or a calibration-specific analysis would be needed to support that stronger conclusion (metric reference).

3 Mathematical Background

3.1 Supervised Learning and Regularization

Given training examples $\{(x_i, y_i)\}_{i=1}^n$, the goal is to learn a predictor that performs well on unseen examples, rather than only minimizing training error. A regularized empirical objective has the form

$$\min_\theta \ \frac{1}{n}\sum_{i=1}^{n}\ell(f_\theta(x_i), y_i) + \lambda\Omega(\theta).$$

The models differ both in their representations and in how they control complexity. The SVM uses a fixed feature map and a margin penalty; the CNN learns features jointly with the classifier and uses weight decay, dropout, batch normalization, and an early-stopping criterion.

3.2 Support Vector Machines

For labels $\tilde{y}_i = 2y_i - 1 \in \{-1, +1\}$, the soft-margin SVM solves

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\xi_i \\ \text{subject to} \quad & \tilde{y}_i(w^\top\phi(x_i) + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0. \end{aligned} \tag{1}$$

The corresponding hinge loss is $\max\{0, 1 - \tilde{y}_i f(x_i)\}$. Larger C penalizes margin violations more strongly, while smaller C permits more violations in exchange for stronger regularization.

The kernel formulation avoids explicitly constructing ϕ. Its dual is

$$\begin{aligned} \max_\alpha \quad & \sum_i \alpha_i - \frac{1}{2}\sum_{i,j}\alpha_i\alpha_j\tilde{y}_i\tilde{y}_j K(x_i, x_j) \\ \text{subject to} \quad & 0 \leq \alpha_i \leq C, \qquad \sum_i \alpha_i\tilde{y}_i = 0. \end{aligned} \tag{2}$$

The code uses the radial basis function kernel

$$K(x, z) = \exp(-\gamma\|x - z\|^2).$$

Parameter γ controls the locality of the kernel response. The decision score is a weighted combination of kernel similarities to support vectors, plus an intercept. Probability estimates are obtained by enabling `SVC(probability=True)`, which adds an internal probability-fitting procedure (SVC reference).

3.3 PCA Features

Each image is flattened into a vector of 784 pixel values. A `StandardScaler` is fitted on the training vectors, and PCA is then fitted to the standardized training data. If X is centered and $\Sigma = X^\top X/(n-1)$, PCA finds the leading eigenvectors of Σ and projects X onto them:

$$Z = XW_k, \qquad \frac{\sum_{j=1}^{k}\lambda_j}{\sum_j \lambda_j} \geq 0.95.$$

The implementation retains enough components to explain 95% of variance. The number of components actually retained was not recorded in the slides, so no fixed numerical dimension is claimed here.

PCA provides a compact linear representation of intensity variation. It does not explicitly encode local edges or learn features specialized for the class labels.

3.4 HOG Features

HOG describes local edge orientation. Image gradients define magnitudes and angles, which contribute to orientation histograms within spatial cells. Histograms from neighboring cells are normalized in overlapping blocks and concatenated into one feature vector.

The project uses 9 orientation bins, 4×4 pixels per cell, and 2×2 cells per block. At 28×28 resolution this yields 7×7 cells, 6×6 overlapping blocks, and

$$6 \cdot 6 \cdot 2 \cdot 2 \cdot 9 = 1,296$$

features. These descriptors are standardized before SVM training. HOG encodes local structure, but it does not reduce dimensionality in this configuration: its output is larger than the original 784-pixel vector.

3.5 Compact Convolutional Network

The CNN learns its representation directly from the normalized images. Convolution uses local receptive fields and shared weights; ReLU introduces nonlinearity, and pooling reduces spatial resolution. Shared convolutional weights provide translation equivariance, while pooling can improve tolerance to small local shifts; full translation invariance is not guaranteed.

Operation	Output
Input	$1 \times 28 \times 28$
Conv $1 \to 16$, BN, ReLU, MaxPool	$16 \times 14 \times 14$
Conv $16 \to 32$, BN, ReLU, MaxPool	$32 \times 7 \times 7$
Conv $32 \to 64$, BN, ReLU	$64 \times 7 \times 7$
Global average pooling	64
Dropout ($p = 0.3$), Linear $64 \to 1$	1 logit

Table 2: CNN architecture. Every convolution has a 3×3 kernel and padding 1; both max-pooling operations use 2×2 windows.

The model has $23,585$ trainable parameters. Global average pooling averages each 7×7 feature map, yielding 64 features rather than flattening $3,136$ activations. The final layer returns a logit z, not a probability; $\sigma(z)$ is applied during evaluation.

Binary cross-entropy. Training uses `BCEWithLogitsLoss`, combining sigmoid and binary cross-entropy in a numerically stable expression:

$$\ell(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}).$$

The stable logit formulation avoids explicitly taking logarithms of extreme sigmoid probabilities during training.

Regularization. Batch normalization normalizes activations using mini-batch statistics during training and running estimates at evaluation. Dropout randomly suppresses features during training and is disabled at evaluation. Weight decay is set to 10^{-4}. Early stopping monitors validation AUC with patience 5. These mechanisms address different aspects of optimization and generalization; the experiment does not isolate their individual contributions.

3.6 SGD, Momentum, and Adam

For a mini-batch B_t, a stochastic gradient estimate is

$$g_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_\theta \ell_i(\theta_t).$$

Momentum accumulates a running direction, schematically

$$v_t = \mu v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta_t v_t,$$

with $\mu = 0.9$. The SGD implementation also includes weight decay and a `StepLR` schedule, halving the learning rate every ten epochs.

Adam maintains first- and second-moment estimates:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ \hat{m}_t &= m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t), \\ \theta_{t+1} &= \theta_t - eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \end{aligned}$$

The code uses Adam's default moment coefficients and an initial learning rate of 10^{-3}, without a scheduler. It uses `Adam`, not `AdamW`; identical numerical weight-decay settings do not imply identical optimization trajectories across the two optimizers.

The classical step-size conditions $\sum_t \eta_t = \infty$ and $\sum_t \eta_t^2 < \infty$ motivate stochastic approximation under additional assumptions. They do not establish convergence to a global optimum for this nonconvex CNN, and are not the schedule used in the experiment. A fixed seed makes a random sequence repeatable; it does not remove stochastic gradients or inherently improve an optimizer.

4 Python Implementation

The project is organized into eight blocks in `pneumonia.py`. The original file is retained in `archive/pneumonia_original.py`; the publication entry point adds explicit run settings and the checkpoint correction described below.

Block 1 — Configuration. Imports NumPy, scikit-learn, scikit-image, MedMNIST, Matplotlib, and PyTorch. The device is CUDA when available and CPU otherwise. The supplied script does not select Apple's MPS backend. cuDNN deterministic behavior is requested and benchmarking disabled, but the original seed-setting lines are commented out.

Block 2 — Dataset loading. `get_pneumonia_loaders()` constructs the three official splits. Training data are shuffled; validation and test data are not. `loaders_to_numpy()` converts batches into $(N, 28, 28)$ arrays and one-dimensional labels for the SVM experiments.

Block 3 — PCA and SVM. `compute_pca_features()` performs flattening, standardization, and PCA. `train_eval_svm()` searches over

$$C \in \{0.1, 1, 10\}, \qquad \gamma \in \{10^{-3}, 10^{-2}, 10^{-1}\}.$$

There are nine parameter pairs and five CV folds: at least 45 candidate fits per SVM representation, followed

by refitting. Probability estimation adds its own internal fitting cost. `GridSearchCV` uses ROC AUC scoring and `n_jobs=-1`.

Block 4 — HOG. `compute_hog_features()` extracts descriptors independently for each image. A training-fitted scaler transforms the HOG vectors before the same SVM grid search is applied. Unlike a statement in slide 27, the separate validation set is evaluated after this search; it does not choose the best C, γ pair.

Block 5 — CNN. `SmallPneumoniaCNN` implements Table 2. The forward pass ends in a squeezed vector of logits, one per image. The network is trained from scratch.

Block 6 — Training and evaluation. `train_one_epoch()` computes the mean training loss; `collect_probs_and_labels()` runs the model in evaluation mode; `binary_classification_metrics()` computes the four metrics. `train_cnn()` monitors validation AUC and stops after five epochs without improvement.

Checkpoint correction. The original assignment `best_state = model.state_dict()` retains references to model tensors, which can be modified by later training. It does not reliably preserve the best validation checkpoint (PyTorch guidance). The publication version uses:

```
if metrics_val["auc"] > best_val_auc:
    best_val_auc = metrics_val["auc"]
    best_state = copy.deepcopy(
        model.state_dict())
    patience_counter = 0
```

Listing 1: Preserving the selected checkpoint

This corrects future runs; it does not retroactively change or validate the historical slide results. The original outputs cannot establish which epoch was ultimately evaluated without the missing training artifacts.

Block 7 — Learning curves. Plots training loss and validation AUC against epoch. The publication version also saves PNG plots when plotting is enabled. The report's historical curves were extracted directly from the original PowerPoint during preparation, without reconstructing unavailable numerical histories. The published sources include the slide PDF and extracted PNGs; the editable PowerPoint is not bundled.

Block 8 — Experiment orchestration. Runs both SVM pipelines and both CNN optimizers, records durations, saves metrics and weights, and optionally displays plots. The publication CLI adds `-seed`, `-sgd-lr`, `-sgd-epochs`, `-adam-epochs`, `-output-dir`, and `-no-plots`. The default SGD settings retain the supplied main script's lower learning rate of 0.01 and maximum of 40 epochs; Adam defaults to 30 epochs.

5 Experimental Setup

The presentation describes three experiment groups. Experiment 1 compares SVM features and unseeded CNN runs with SGD learning rate 0.05. Experiment 2 reports seeded CNN runs at the same SGD learning rate. Experiment 3 lowers SGD's learning rate to 0.01 without fixing the seed. Both SGD configurations use momentum; there is no separately documented momentum-free benchmark in the recorded result tables.

Setting	SGD	Adam
Learning rate	0.05 or 0.01	0.001
Momentum	0.9	default moments
Weight decay	10^{-4}	10^{-4}
Batch size	64	64
Maximum epochs	30 or 40	30
Scheduler	StepLR $(10, 0.5)$	none
Early stopping	AUC, patience 5	AUC, patience 5

Table 3: Settings established by the supplied source. Historical per-run configurations are not fully archived.

Evidence and precision. Tables in this report transcribe the presentation values, preserving the available decimal places. AUC entries originally marked with an approximation symbol remain approximate; all displayed metrics are rounded. The tables are not recomputed from raw predictions, and no new full training run is claimed. No confidence intervals, repeated-seed distributions, or saved raw prediction files were supplied.

The local environment contains Python 3.9 and package metadata including PyTorch 2.8.0, torchvision 0.23.0, MedMNIST 3.0.2, scikit-learn 1.6.1, and scikit-image 0.24.0. This is a supplied environment snapshot, not proof of the exact software versions used for every historical result. The hardware behind the reported timings is not specified.

6 Reported Results

6.1 SVM: PCA versus HOG

Model	AUC	Acc.	F1	Brier
SVM + PCA	∼0.947	0.861	0.899	0.112
SVM + HOG	∼0.955	0.862	0.900	0.115

Table 4: Historical SVM results, slide 33 (repeated in slide 36).

HOG improves reported AUC by about 0.008, while accuracy and F1 differ by only 0.001. PCA has the lower Brier score (0.112 versus 0.115), so HOG does not dominate every metric. These observations support HOG as a useful structural representation, without showing that it is uniformly superior.

6.2 CNN: Unseeded and Seeded Runs

Seed	Model	AUC	Acc.	F1	Brier
Off	SGD	0.9558	0.705	0.809	0.237
Off	Adam	0.9579	0.822	0.875	0.143
On	SGD	0.9660	0.888	0.907	0.078
On	Adam	0.9644	0.893	0.917	0.075

Table 5: Presentation-reported CNN results (slides 34 and 37). SGD uses learning rate 0.05. AUC entries are approximate.

For the unseeded setting, Adam has better reported values on all four metrics than SGD. The unseeded SGD run has strong ranking AUC but low accuracy and a high Brier score. This illustrates that good ranking need not imply reliable probabilities or an effective fixed decision threshold.

Both seeded runs improve on the displayed unseeded runs. Seeded SGD has the highest reported AUC (0.9660), whereas seeded Adam has the best accuracy (0.893), F1 (0.917), and Brier score (0.075). However, one seeded and one unseeded run per optimizer do not establish a causal benefit of fixing the seed, or a general ranking of optimizer stability.

6.3 SGD Learning-Rate Variation

LR	Seed	AUC	Acc.	F1
0.05	Off	0.9558	0.705	0.809
0.01	Off	0.9491	0.758	0.838
0.05	On	0.9660	0.888	0.907

Table 6: SGD comparison transcribed from slide 39. Brier score is not reported for the 0.01 unseeded run.

Lowering the learning rate improves reported accuracy from 0.705 to 0.758 and F1 from 0.809 to 0.838, but AUC falls from 0.9558 to 0.9491. It therefore does not improve all evaluation criteria. The lower-learning-rate run also differs in its random trajectory and may differ in training duration; the supplied main script allows 40 rather than 30 epochs in that setting. The results do not isolate a pure learning-rate effect.

The original curves in Figures 2–4 show falling training loss together with fluctuating validation AUC. In particular, the seeded SGD plot contains a large early validation drop, so the presentation's stability statements should be read as qualitative impressions, not a statistical estimate of variance across runs.

6.4 Runtime Comparison

Model	Reported time
SVM + HOG	approximately 410 s
CNN + SGD, seeded	approximately 40 s
CNN + Adam, seeded	approximately 25–40 s

Table 7: Approximate timings from slide 41. Hardware and exact historical timing boundaries were not recorded.

In the supplied source, SVM timing includes feature extraction, the grid search, and validation/test evaluation. CNN timing includes loader construction, training, and evaluation for one optimizer configuration. The search budgets are therefore different. The reported timing advantage cannot be interpreted as a general hardware-independent speedup of CNNs over SVMs, although it is consistent with this small CNN being inexpensive to train.

7 Discussion and Limitations

7.1 What the Results Support

The best displayed CNN configurations exceed the SVM baselines in AUC, accuracy, F1, and Brier score. Learned spatial representations are a plausible reason for this outcome, but representation choice is not the only difference: optimization, regularization, hyperparameter budgets, and model selection also vary. The result is evidence for these particular configurations on this benchmark, rather than proof that CNNs always outperform handcrafted features.

The ranking differs by metric. SVM with HOG has higher AUC than SVM with PCA, but a slightly worse Brier score. Seeded SGD has the highest recorded CNN AUC, while seeded Adam performs best on the remaining displayed metrics. The unseeded CNN runs do not uniformly outperform the SVMs.

7.2 Preprocessing and Cross-Validation

Both scalers and PCA are fitted on the entire training split before `GridSearchCV`. Validation and test images are not used to fit these transformations. However, each CV held-out fold has already influenced the preprocessing fitted on the complete training split. This creates cross-validation leakage that can bias model selection (preprocessing guidance).

A stricter future evaluation should place the scaler and PCA inside a scikit-learn `Pipeline`, allowing them to

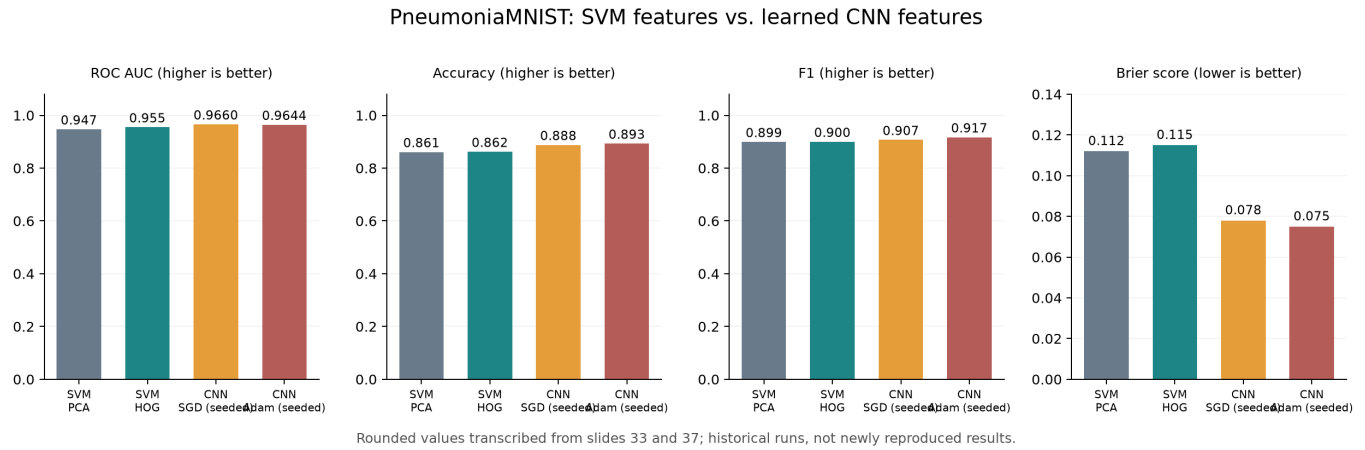


Figure 1: Final model comparison reconstructed from the presentation tables. CNN entries refer to the reported seeded runs. Values are rounded and no error bars are available.

be refitted separately in each training fold. HOG can be computed image by image outside CV, but its scaler should still be fitted within each fold. This would change the experiment and requires a fresh evaluation; the publication script retains the original preprocessing order and documents this limitation.

7.3 Randomness and Model Selection

The original seed-setting block is commented out, so executing the supplied file does not automatically reproduce the presentation's seeded results. The SVM optimization problem is comparatively stable, but `probability=True` uses randomized probability fitting and the training loader shuffles the input order. It is therefore too strong to call the entire probabilistic SVM pipeline strictly deterministic (SVC reference).

The publication CLI explicitly seeds Python, NumPy, PyTorch, CUDA, and the SVM probability-fitting procedure when `-seed` is supplied. The exact historical seed is not recorded in the result tables; 42 appears only in the commented source and is an example for new runs. Repeated experiments across several seeds are needed to estimate variability. A single favorable seed cannot establish optimizer robustness.

The checkpoint-copy correction makes future best-validation selection work as intended. Because it changes the model potentially evaluated on test, new results must be distinguished from the old presentation values.

7.4 Calibration, Imbalance, and External Validity

The class imbalance and the difference in class proportions across splits can affect thresholded metrics and predicted probabilities. The available aggregate scores do not identify the cause of individual errors or prove that a specific calibration mechanism explains the observed differences. Confusion matrices, reliability diagrams, and threshold analysis would make future evaluation more informative.

No patient-level audit beyond the provided dataset split, external validation, confidence intervals, or subgroup analysis is supplied. The images are low-resolution benchmark inputs. Consistent with MedMNIST's stated scope, these results do not establish clinical diagnostic performance (MedMNIST).

8 Reproducing and Extending the Project

Create a fresh environment and install the supplied snapshot, then run the publication entry point:

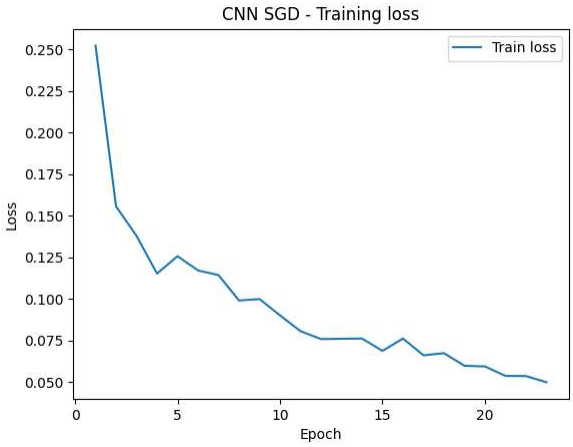
```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements-reproduction.txt
python pneumonia.py --seed 42 \
    --sgd-lr 0.05 --sgd-epochs 30 \
    --output-dir results_runs/seed42 \
    --no-plots
```

Listing 2: Example seeded run

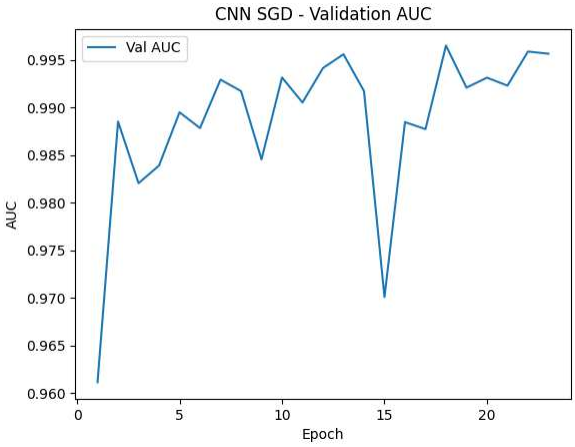
The dataset is downloaded by MedMNIST on first use. A fresh installation with Python 3.11 is a practical target for the recorded dependency versions, but the snapshot is not a validated historical lockfile.

New runs save metric JSONs, CNN weights, timing information, and `run_config.json`. Plotting-enabled runs also save training-loss and validation-AUC PNGs. The historical results in `results/` remain separate: they are CSV transcriptions of slide tables, with source-slide references and a figure provenance manifest. The raw historical prediction arrays and training-history JSONs are not available.

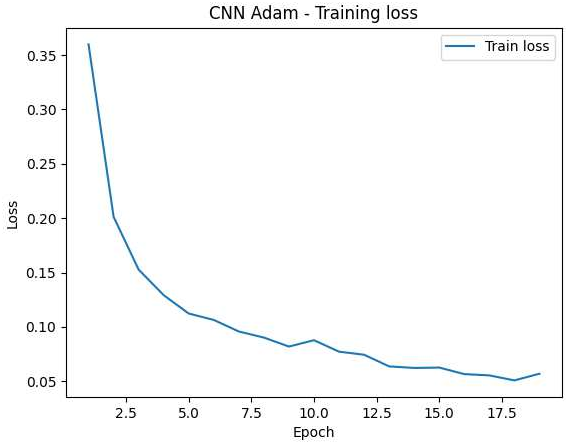
Recommended extensions are: preprocessing inside CV folds, repeated seeded runs with confidence intervals, consistent training/search budgets, reliable checkpoint selec-



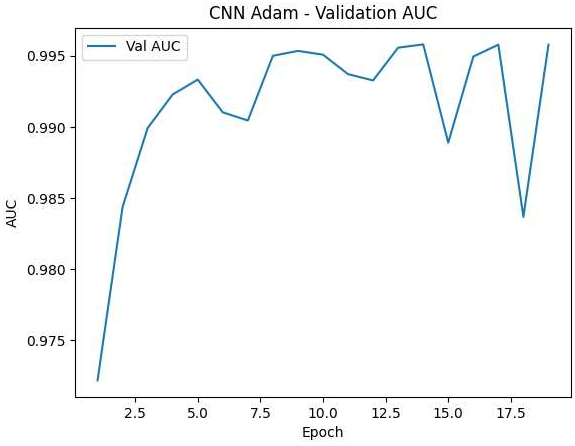
(a) SGD: training loss



(b) SGD: validation AUC

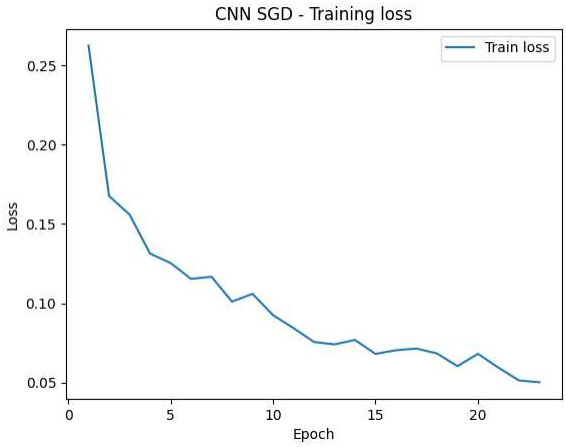


(c) Adam: training loss

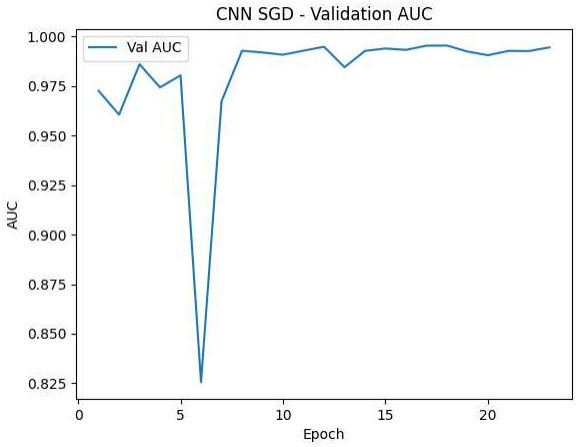


(d) Adam: validation AUC

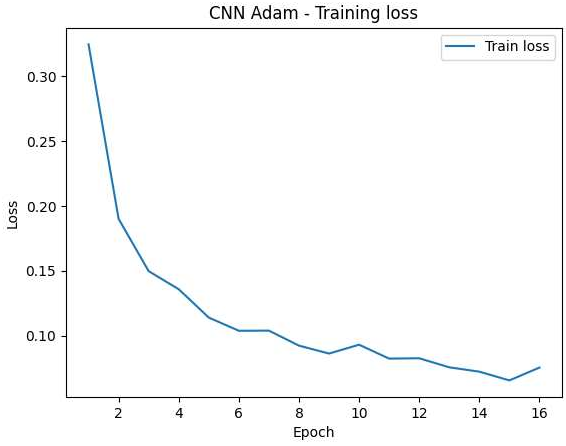
Figure 2: Experiment 1: original unseeded learning curves extracted from slide 35. SGD learning rate is 0.05. The curves describe training and validation, not the final test AUC values in Table 5.



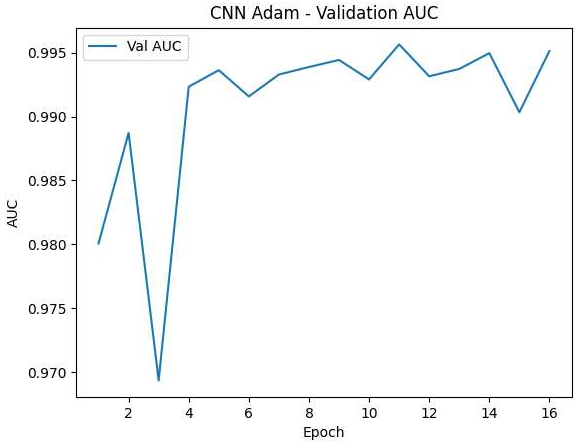
(a) SGD: training loss



(b) SGD: validation AUC

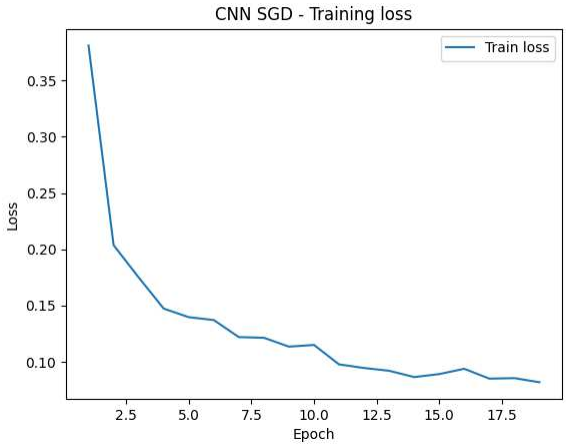


(c) Adam: training loss

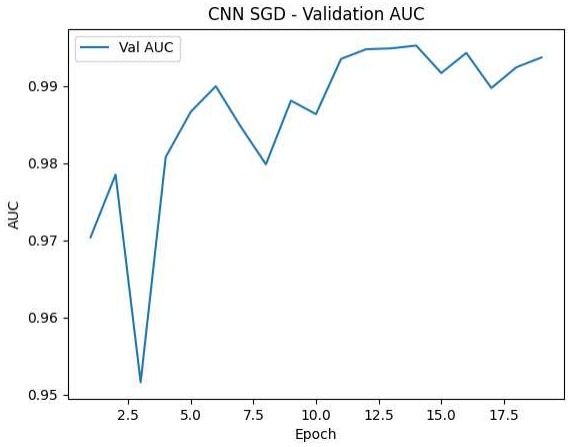


(d) Adam: validation AUC

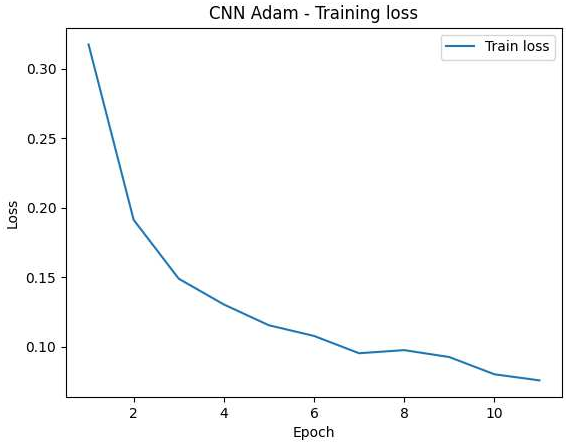
Figure 3: Experiment 2: original seeded learning curves from slide 38. The pronounced early dip in the displayed SGD validation AUC is retained; these curves do not support a claim of uniformly smooth seeded optimization.



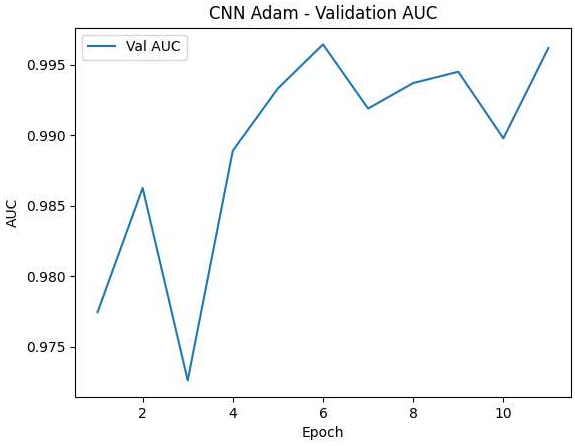
(a) SGD: training loss



(b) SGD: validation AUC



(c) Adam: training loss



(d) Adam: validation AUC

Figure 4: Experiment 3: curves extracted from slide 40. SGD uses learning rate 0.01 without a fixed seed. The slide also contains the Adam curves shown here, but supplies no separate numeric Adam result table for this experiment.

tion, and calibration/threshold diagnostics. These extensions would strengthen the conclusions without changing the educational comparison between explicit features and learned representations.

9 Conclusions

PneumoniaMNIST provides a compact setting for connecting statistical-learning theory with practical model comparison. The SVM pipelines establish strong baselines, with HOG improving discrimination over PCA in the reported runs. The compact CNN achieves the best displayed results when trained in the seeded configurations: SGD leads on AUC and Adam on accuracy, F1, and Brier score.

The optimizer experiments also show why several metrics are needed: lowering SGD's learning rate improves some thresholded scores while reducing AUC, and high AUC can coexist with poor probabilities. The available data do not support universal claims about seeds, calibration, or runtime superiority.

The report preserves the presentation evidence while making implementation limitations explicit. A corrected checkpoint snapshot and explicit execution settings improve the publication code, but a repeated and carefully controlled evaluation is required before stronger empirical conclusions can be drawn.